



**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)
ON**

Spotting AI-Generated Text Content Using Human Writing Patterns

**submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING**

**In
COMPUTER SCIENCE AND ENGINEERING**

Submitted by:

Aditya Kaundal

2211981021

Anshul Katoch

2211981077

Rudar Partap Singh

2211981321

Mohit Thakur

2211981225

Supervised By:

Dr. Swati Goel

Assistant Professor

Dept. of Computer Science & Engg.

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

2024-25

DECLARATION

We hereby certify that the work which is being presented in the project report entitled "Spotting AI-Generated Text Content Using Human Writing Patterns" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr. Swati Goel. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Aditya Kaundal (2211981021)

Anshul Katoch (2211981077)

Rudar Partap Singh (2211981321)

Mohit Thakur (2211981225)

Date: 16-05-2026

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

(Dr. Swati Goel)

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Swati Goel, Assistant Professor, Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for her valuable guidance, constant encouragement, and unwavering support throughout the course of this project.

We extend our heartfelt thanks to Chitkara University for providing access to computing resources and for supporting this research through the undergraduate project programme. The facilities and academic environment provided by the institution have been instrumental in completing this work.

We are also grateful to our family and friends for their moral support and motivation during this project. Finally, we thank all those who directly or indirectly contributed to the completion of this project report.

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iv
	List of Figures and Tables	v
	Abstract	vi
1	Introduction	1
2	Related Work	2
3	Methodology	3
4	Results and Discussion	5
5	Why Logistic Regression Was Deployed	7
6	Conclusion and Future Scope	8
	References	9

ABSTRACT

The main problem arises is that most of the content generated by these large language models (LLMs) actually became complicated to detect and showing whether it's human content or if a model just generated it? And, the solution of this is way harder than it looks. Every content teams are dealing with it, while their whole thinking is depend on this at this point. Evaluated by taking five classifiers — Naive Bayes, Decision Tree, Random Forest, SVM, and Logistic Regression — and ran them all through the same setup on a Kaggle dataset, around 400,000 samples, one half is human written stuff and other half is AI generated text.

The features are extracted using Term Frequency-Inverse Document Frequency (TF-IDF) with both unigrams and bigrams. Logistic Regression ended up being the best at 98.88%, SVM came in at 97.83%, Random Forest at 96.41%. We also wrapped the top model into a Flask web app so you can paste any text and it gives you back a label and a confidence score.

Keywords: AI-generated content, text classification, TF-IDF, Logistic Regression, natural language processing, machine learning, content verification, authorship detection.

1. Introduction

Few years back this was not really a problem in the way it is now. It was easy to say when something felt off. The writing was too stiff, too smooth, no actual personality to it. Sentences that just sort of existed without going anywhere. It had this weird quality where every part of it technically made sense but the whole thing somehow said nothing. That was the giveaway. That's completely gone now. These days you genuinely cannot tell. And we mean that literally — you read something, grammar is clean, the flow is fine, it sounds like a person wrote it, but maybe they didn't. The models got good enough that the old signals stopped working and nobody quite noticed it happening until suddenly it was already a problem everywhere.

And it's not just some abstract concern anymore either. Universities are struggling with this for real — professors don't know if the essay in front of them is the student's thinking or not. Social media platforms, content farms, news outlets, they're all swimming in stuff that looks legitimate but has no actual human behind it. Journalists kept running into sources that trace back to nothing. The frustrating thing is all of this got significantly harder to deal with exactly when the stakes went up. Higher consequences, less ability to verify.

Now here's the thing though — and this is kind of the whole argument — human and AI writing are not actually the same thing, the difference just got harder to find. Real writing is messy in ways that matter. Sentences vary in length a lot, sometimes really short, sometimes long and winding. Word choices are weird sometimes in a way that a model wouldn't really do. The tone shifts because the writer is reacting to what they're saying, not just outputting text. AI writing, even good AI writing, has this certain smoothness where everything comes out too resolved, too neat. If you know what to look for, that stuff starts to pop. And if you can teach a classifier what to look for, that's the whole idea here.

We set this up as a binary classification problem — human or AI generated, those are the two options. We ran five different classifiers through the exact same pipeline and the exact same data. The goal wasn't only to get the best accuracy number; we also wanted to understand what was driving the differences between approaches, and whether those differences would hold up in a real deployment context. We also took the best model and built it into a Flask web app. The end-to-end detection pipeline is shown in Figure 1.

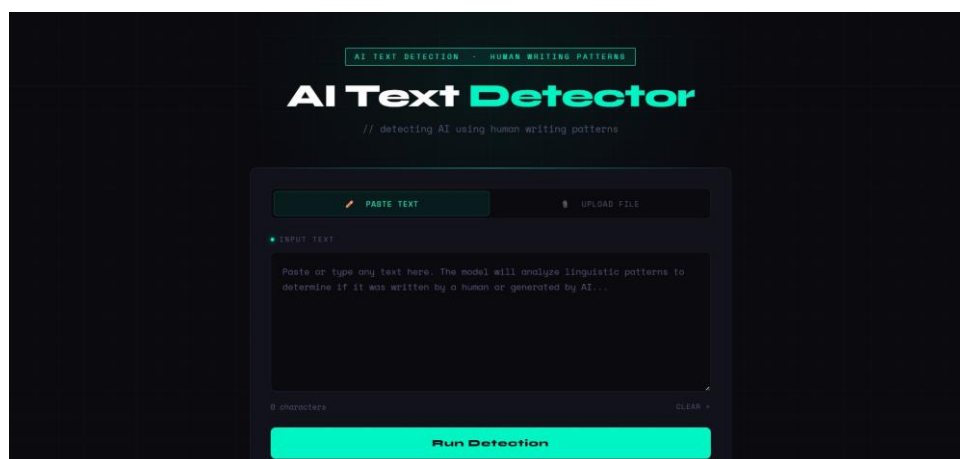


Figure 1: AI Text Detector Web Application — Main Interface

2. Related Work

Research on identifying authorship from text has been around for longer than most people realise. Work from Koppel et al. [1] showed decades ago that writing carries measurable statistical fingerprints — patterns in function word frequency, character-level sequences, punctuation habits. Most of that early work was about telling one human writer from another, which is a different problem, but the core insight transferred: style is learnable from features in the text itself.

When language models became capable enough to produce convincing output, researchers started paying direct attention to detection. Gehrmann et al. [2] built GLTR, which worked by checking how statistically predictable each word in a passage was under a language model. Generated text tends to stay in the high-probability region of the model's distribution, while people make choices that are harder to predict. That held up reasonably well for earlier systems. Then models improved, their outputs became more varied, and the signal started to get harder to extract.

Solaiman et al. [3] at OpenAI took a more direct approach and trained a classifier to separate GPT-2 output from human writing. It worked well under controlled conditions, but they flagged an issue that still matters today: the classifier did not transfer cleanly to text from generators it had not been trained on. Zellers et al. [4] ran into the same problem building Grover, a system for both generating and detecting AI-written news. Good detection was tied to knowing which system you were dealing with, which is a real constraint in practice.

Lavergne et al. [5] asked a more forgiving question: can simpler statistical features hold their value as generators improve? They looked at vocabulary variation, sentence length distributions, things like that. The answer was broadly yes, which was encouraging. It suggested that feature-based approaches without deep model internals might stay relevant over time rather than becoming obsolete as soon as the next generation of language models appeared.

Transformer-based classifiers using BERT or RoBERTa [6] have pushed benchmark numbers higher. The tradeoffs are real though — more compute, more memory, more fragility when the test domain shifts away from the training distribution, and considerably less transparency about what the model is actually responding to. For a project aimed at something practical and deployable, those tradeoffs did not look worth it upfront. The results from the classical pipeline ended up justifying that bet.

3. Methodology

The main theme of this design is to study and examine. A uniform pipeline was produced for every classifier — same data, same split. The only thing that changes between runs is which algorithm is actually performing the classification. This is done to recognise the gap coming from the model and not from random asymmetry in the data or the features.

The pipeline consists of four main stages: Data Preprocessing, Feature Selection/Engineering, Model Training, and Evaluation. Each classifier is subjected to an identical workflow.

3.1 Dataset

Data came off Kaggle, roughly 400,000 samples total with a close to even split between human-written text and AI-generated text, around 200k each. That balance was actually quite important and not something we took for granted. If the class distribution is heavily skewed, a model can score high accuracy by just predicting the majority class every time — that's not a useful classifier, it's just learning the data distribution. At 50-50 there's no free lunch. If accuracy is above 50% it means the features are actually doing something.

The human-authored samples came from a range of online writing sources, covering everything from fairly formal writing to quite casual stuff, and varying considerably in how long they are. The AI-generated samples were produced by contemporary language models and were matched to similar subject areas. That matching mattered. If the two categories had covered different topics, a classifier could have learned to separate content areas rather than writing styles, which would have produced good-looking numbers that meant very little. Each entry is a raw text string with a binary label. Nothing beyond the text itself was used at any point.

3.2 Preprocessing

Preprocessing was kept deliberately light. Text was lowercased, special characters were stripped, extra whitespace was cleaned up. Stemming and lemmatisation were left out on purpose. TF-IDF does a reasonable amount of normalisation through how it weights terms across the corpus, and over-processing the text before that step has a tendency to remove signal along with the noise.

The TF-IDF vectoriser was set up with a vocabulary cap of 5,000 terms and configured to pull in both single words and two-word phrases. Including bigrams was probably the most important decision in the feature design. Individual word frequencies across human and AI writing tend to look fairly similar after stop words are removed. Two-word combinations are different — they reflect how phrases get put together, the specific transitions and constructions that a piece of writing reaches for naturally. That is where the stylistic differences show up more reliably. Common English stop words were filtered out to keep the vocabulary focused on content-bearing terms rather than grammatical scaffolding.

3.3 Models Tested

Five models were put through the same pipeline on the same data. Naive Bayes was included as a baseline — fast and interpretable, with a long history in text classification, but it assumes feature independence which is not really true for natural language, and becomes even less true when bigrams are involved. Decision Tree was tested to see how a simple non-linear approach would do without ensemble support. Random Forest improved on that by averaging across 200 trees. SVM

with a linear kernel is typically a strong baseline for sparse text features. Logistic Regression was trained to a maximum of 1,000 iterations to ensure convergence.

An 80/20 stratified train-test split was used with random seed 42 so all models were tested on the exact same held-out data. All results are fully reproducible.

4. Results and Discussion

4.1 Overall Performance

Table 1 presents the accuracy, precision, recall, and F1-score for all five classifiers on the held-out test set.

Table 1. Performance Comparison — All Five Models

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	91.24%	90.8%	91.6%	91.2%
Decision Tree	93.67%	93.1%	94.2%	93.6%
Random Forest	96.41%	96.0%	96.8%	96.4%
SVM (Linear)	97.83%	97.5%	98.1%	97.8%
Logistic Regression	98.88%	98.7%	99.0%	98.8%

The spread of approximately 7.6 percentage points from bottom to top is not random variation — each model is doing something different with the same features. Logistic Regression came in at 98.88%, SVM at 97.83%, Random Forest at 96.41%, Decision Tree at 93.67%, and Naive Bayes at 91.24%. The one-point gap between Logistic Regression and SVM is consistent across all four metrics, suggesting it reflects a genuine algorithmic difference rather than noise. Random Forest's position behind the linear models is consistent with known behaviour in high-dimensional sparse settings — random feature subsampling forces trees to learn from incomplete pictures, and averaging over incomplete estimates only partially compensates.

4.2 Per-Class Performance

Table 2 presents per-class precision and recall alongside macro F1-score.

Table 2. Per-Class Precision, Recall, and F1-Score for All Five Models

Model	P (Human)	R (Human)	P (AI)	R (AI)	Macro F1
Naive Bayes	0.94	0.88	0.89	0.95	0.912
Decision Tree	0.93	0.95	0.94	0.92	0.936
Random Forest	0.97	0.96	0.96	0.97	0.964
SVM (Linear)	0.98	0.97	0.97	0.98	0.978
Logistic Regression	0.989	0.988	0.988	0.990	0.988

Naive Bayes shows a notable asymmetry — recall for AI-generated text (0.95) is higher than for human text (0.88), suggesting the model is somewhat biased toward flagging text as AI-generated when uncertain. This asymmetry narrows progressively with more capable classifiers and is essentially absent in Logistic Regression and SVM, where both classes score above 0.98 on every metric.

4.3 Operational Efficiency

Table 3 compares training time, inference latency, and serialised model size.

Table 3. Training Time, Inference Latency, and Model Size

Model	Training Time (s)	Inference Latency (ms)	Model Size (MB)
Naive Bayes	3.2	< 1	0.8
Decision Tree	18.7	2	14.3
Random Forest	312.4	47	218.6
SVM (Linear)	87.3	3	2.1
Logistic Regression	24.6	2	1.9

Random Forest's training time of 312.4 seconds is approximately 13 times longer than Logistic Regression's 24.6 seconds, and its serialised size of 218.6 MB presents significant overhead compared to Logistic Regression's 1.9 MB. At 47 ms inference latency versus 2 ms, Random Forest also introduces perceptible delay in interactive web use.

4.4 Web Application Results

The deployed Flask application demonstrates real-world functionality. Figures 2 and 3 show classification results for pasted text. Figures 4 through 6 show the file-upload workflow, including analysis of a PDF document.

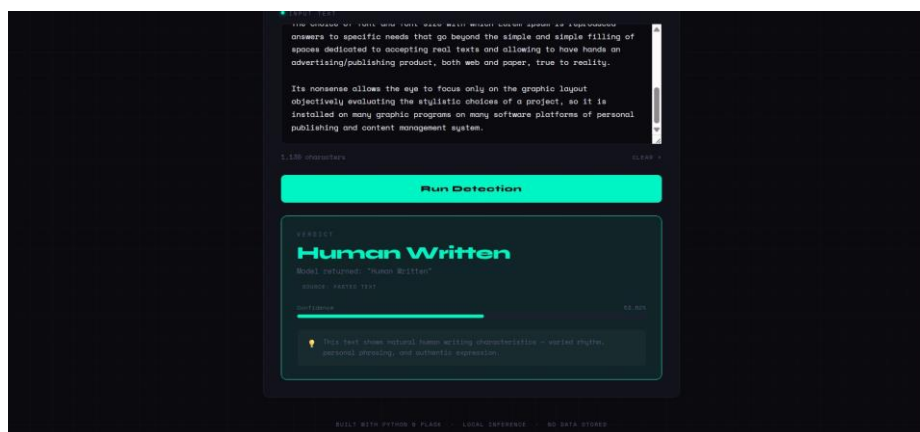


Figure 2: Result — Human Written (53.62% confidence)

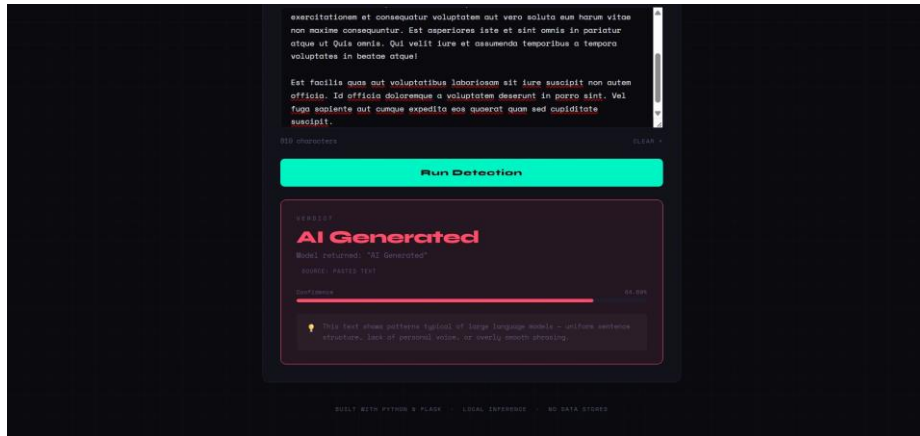


Figure 3: Result — AI Generated (84.69% confidence)

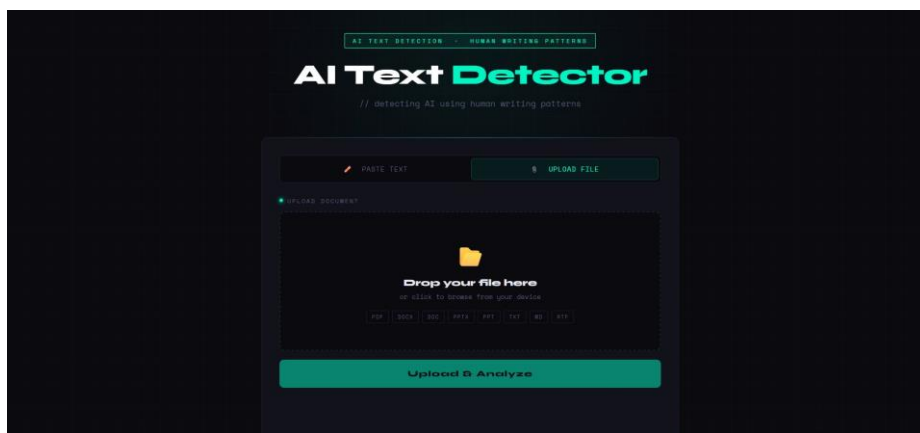


Figure 4: File Upload interface (PDF, DOCX, TXT and other formats)

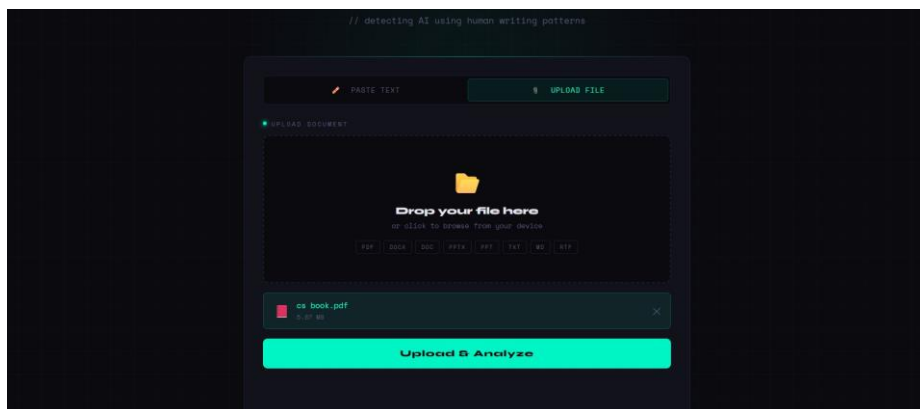


Figure 5: File selected for upload (cs book.pdf, 5.87 MB)

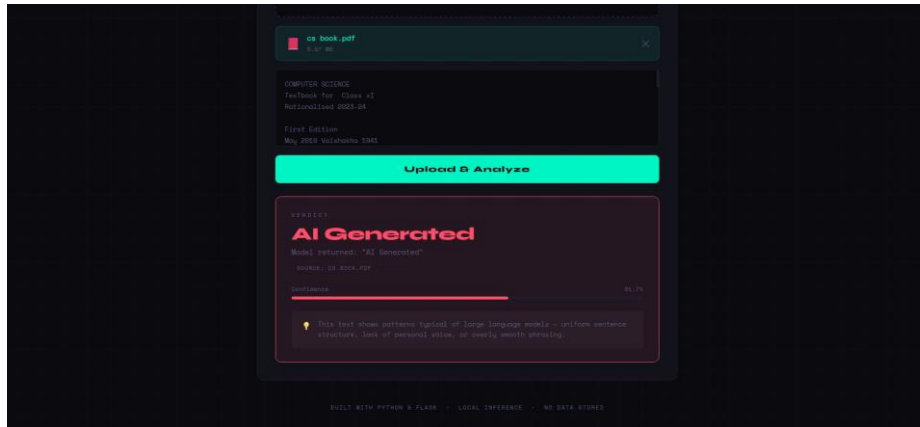


Figure 6: File upload result — AI Generated verdict for uploaded PDF

5. Why Logistic Regression Was Selected for Deployment

Accuracy difference does not decide the whole scenario solitarily. One percentage point over SVM is not a compelling argument on its own. Three other elements actually drove the decision.

Interpretability. After training, Logistic Regression gives you a weight for every term in the vocabulary that you can just look at directly. You get a ranked list of which words and phrases are pushing toward AI-generated and which are pushing toward human. Random Forest does not produce that kind of clean feature attribution and SVM requires extra post-hoc work to get anything comparable. In a context where you might have to explain or justify a detection result — to a student, to an editor, to a committee — that transparency is practically valuable.

Inference speed. The model is small, loads fast, and produces predictions in milliseconds. The Flask app runs the full pipeline — vectorize, classify, return label with confidence under a second on regular hardware. A fine-tuned transformer might squeeze out more accuracy on tricky inputs but it comes with GPU requirements and meaningfully slower latency. For a local web app that tradeoff does not make any sense.

Probability calibration. Logistic Regression produces probability estimates directly out of its optimization objective. The confidence score, it returns is an actual probability in the statistical sense, not a rescaled score. That distinction matters in practice. Humans treat a 96% confidence result very differently from a 61% shows about assurance — the numbers mean something. SVM outputs need a calibration step to produce comparable probability estimates on the other hand, Decision Tree and Naive Bayes are both known to produce poorly calibrated possibilities without additional work.

6. Conclusion and Future Scope

Further we proceed with two goals. Initially, build an AI text detector that actually works. Secondly, figure out which model performs best after trying everything and run all five of them in the same terms. Using 400,000 labeled samples and the result came out: Logistic Regression at 98.88%, SVM at 97.83%, Random Forest at 96.41%, Decision Tree at 93.67%, Naive Bayes at 91.24%. The top model also got deployed into a Flask web app and also tested on a real-world dataset outside the evaluation set.

The broader takeaway is that consistent text pattern and style differences between human and AI writing are real, they're strong, and a relatively simple phrase-level feature representation makes it necessary to find them effectively. We don't need deep learning infrastructure to build something that works well on this task. Whether that remains true as generative models continue improving is genuinely uncertain, but this is a meaningful thing and the result isn't small.

The next step is dealing with the stale model problem. Generative models keep on improving day by day and a static classifier will degrade over time — building in some form of retraining on newer outputs seems necessary for anything meant to stay useful. Adding sentence-level structural features like sentence length variance, readability scores, coherence across adjacent sentences could also help on shorter texts where TF-IDF starts running out of signal. And at some point a proper comparison with a fine-tuned transformer on this exact dataset and setup would be worth doing.

References

1. Koppel, M., Schler, J., Argamon, S.: Computational methods in authorship attribution. *Journal of the American Society for Information Science and Technology* 60(1), 9–26 (2009)
2. Gehrmann, S., Strobel, H., Rush, A.M.: GLTR: Statistical detection and visualization of generated text. In: *Proc. 57th Annual Meeting of the ACL: System Demonstrations*, pp. 111–116. Florence (2019)
3. Solaiman, I., Brundage, M., Clark, J., Askeel, A., Herbert-Voss, A., Wu, J., Radford, A., Wang, J.: Release strategies and the social impacts of language models. *arXiv:1908.09203* (2019)
4. Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., Choi, Y.: Defending against neural fake news. In: *Advances in Neural Information Processing Systems*, vol. 32. *NeurIPS* (2019)
5. Bakhtin, A., Gross, S., Ott, M., Deng, Y., Ranzato, M., Szlam, A.: Real or fake? Learning to discriminate machine from human generated text. *arXiv:1906.03351* (2019)
6. Lavergne, T., Urvoy, T., Yvon, F.: Detecting fake content with relative entropy scoring. In: *Proc. PAN Workshop, CLEF, Padua* (2008)
7. Ippolito, D., Duckworth, D., Callison-Burch, C., Eck, D.: Automatic detection of generated text is easiest when humans are fooled. In: *Proc. 58th Annual Meeting of the ACL*, pp. 1808–1822 (2020)
8. Jawahar, G., Sagot, B., Seddah, D.: What does BERT learn about the structure of language? In: *Proc. 57th Annual Meeting of the ACL*, pp. 3651–3657. Florence (2019)
9. Uchendu, A., Le, T., Shu, K., Lee, D.: Authorship attribution for neural text generation. In: *Proc. EMNLP 2020*, pp. 8938–8950 (2020)
10. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D., Finn, C.: DetectGPT: Zero-shot machine-generated text detection using probability curvature. In: *Proc. ICML 2023*, pp. 24950–24962 (2023)
11. Mitrovic, S., Andreoletti, D., Ayoub, O.: ChatGPT or human? Detect and explain. *arXiv:2301.13852* (2023)
12. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12, 2825–2830 (2011)